UITS

Future will be better than thy past

An initiative of PHP Family

University of Information Technology & Sciences

ASSIGNMENT

Course Title : Data Structures and Algorithms I Lab

Course Code : CSE0613212

Title : Implementation of Selected Problems Jump Game, Find the Different Ones, Search a 2D Matrix II, and Daily Temperatures Using C++.

Submitted to

Teacher's Name : Md. Azharul Karim Chowdhury Anik

Designation : Lecturer,
Department of CSE, UITS

Submitted by

Name : Fahim Hossain

ID : 04325205101037

Department : CSE

Batch : 58th

Section : 3A

Semester : Autumn 2026

Date of Submission: ??-11-2026

Question 1: Jump Game (LeetCode 55).

You are given an integer array **nums.** You are initially positioned at the array's **first index**, and each element in the array represents your maximum jump length at that position.

Return **true** if you can reach the last index, or **false** otherwise.

Example 1:

- **Input:** nums = [2,3,1,1,4]
- **Output:** true
- **Explanation:** Jump 1 step from index 0 to 1, then 3 steps to the last index.

Example 2:

- **Input:** nums = [3,2,1,0,4]
- **Output:** false
- **Explanation:** You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.

Constraints:

- 1 <= nums.length <= 10^4
- 0 <= nums[i] <= 10^5

Code:

Start here | Jump_Game.cpp | Diff_ones.cpp | Search_2D_Matrix.cpp | Daily_Temp.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter array size: ";
    cin >> n;

    int nums[100];

    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }

    int farthest = 0;
    bool result = true;

    for (int i = 0; i < n; i++) {

        if (i > farthest) {
            result = false;
            break;
        }

        if (i + nums[i] > farthest) {
            farthest = i + nums[i];
```

Start here | Jump_Game.cpp | Diff_ones.cpp | Search_2D_Matrix.cpp | Daily_Temp.cpp

```cpp
    int farthest = 0;
    bool result = true;

    for (int i = 0; i < n; i++) {

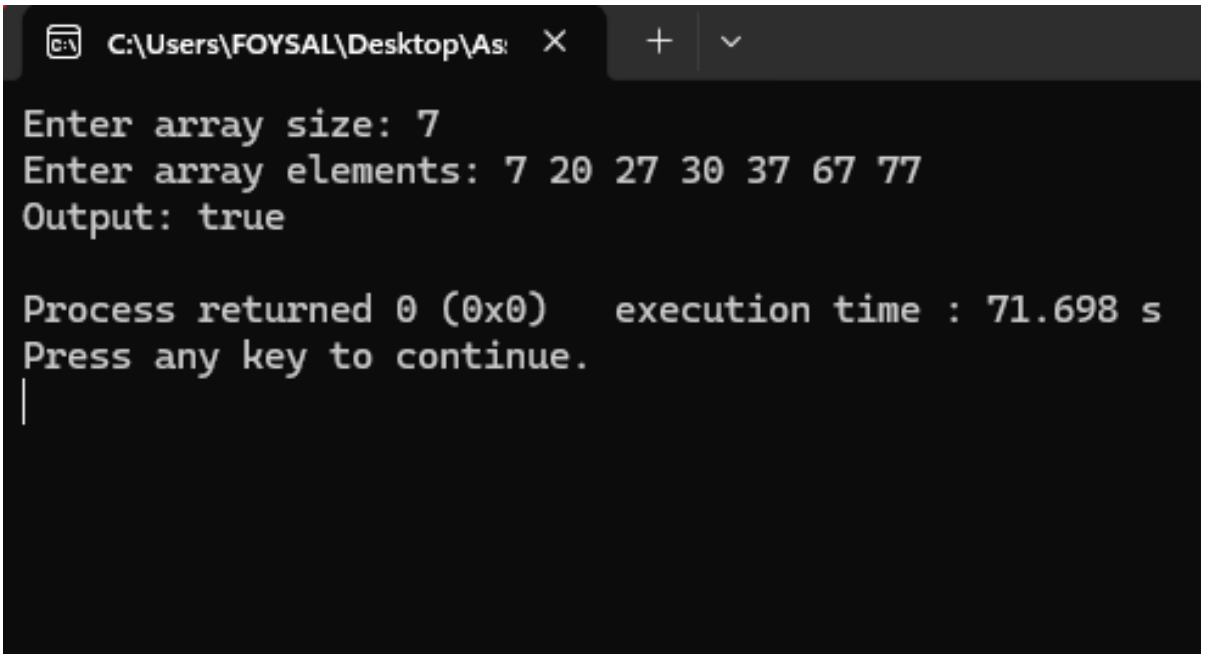
        if (i > farthest) {
            result = false;
            break;
        }

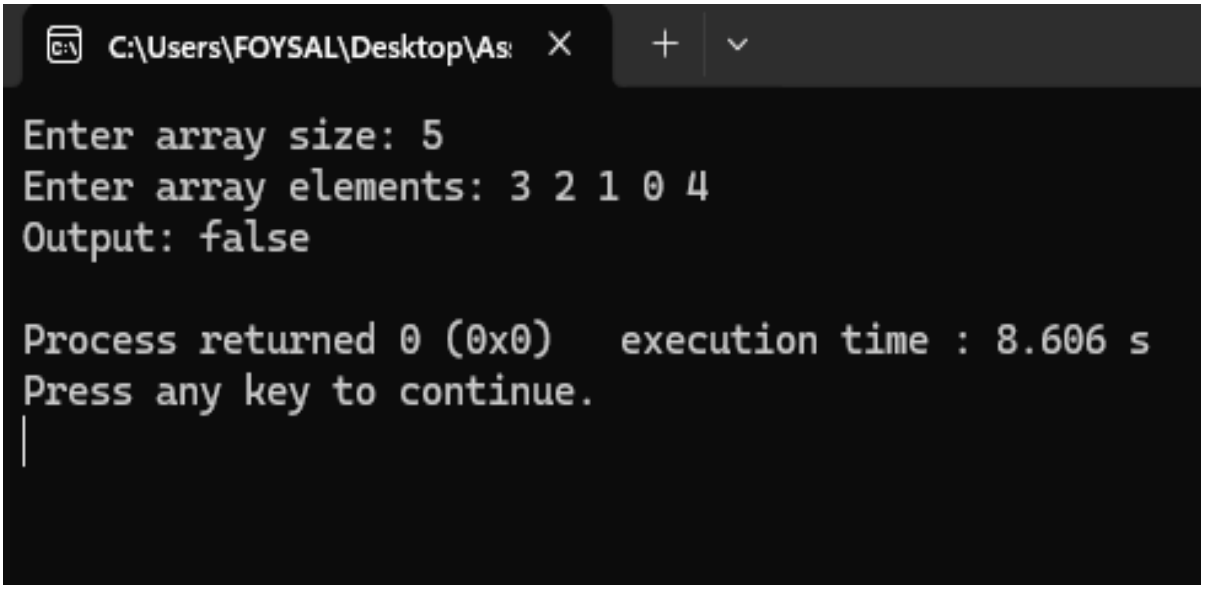
        if (i + nums[i] > farthest) {
            farthest = i + nums[i];
        }
    }

    if (result) {
        cout << "Output: true" << endl;
    }
    else {
        cout << "Output: false" << endl;
    }

    return 0;
}
```

Output:


C:\Users\FOYSAL\Desktop\As
Enter array size: 7
Enter array elements: 7 20 27 30 37 67 77
Output: true
Process returned 0 (0x0)   execution time : 71.698 s
Press any key to continue.


C:\Users\FOYSAL\Desktop\As
Enter array size: 5
Enter array elements: 3 2 1 0 4
Output: false
Process returned 0 (0x0)   execution time : 8.606 s
Press any key to continue.

Question 2: Find the Different Ones! (Codeforces D).

You are given an array a of n integers, and q queries.

Each query is represented by two integers **l** and **r (1 <= l <= r <= n).** Your task is to find, for each query, two indices i and j (or determine that they do not exist) such that:

★ **- l <= i <= r;**

★ **- l <= j <= r;**

★ **- a_i != a_j.**

In other words, for each query, you need to find a pair of different elements among **al, a(l+1), ..., a_r,** or report that such a pair does not exist.

Input:

The first line of the input contains a single integer t **(1 <= t <= 10^4)** — the number of test cases. The descriptions of the test cases follow.

The first line of each test case contains a single integer **n (2 <= n <= 2 * 10^5)** — the length of the array **a**.

The second line of each test case contains n integers **a1, a2, ..., an (1 <= ai <= 10^6)** — the elements of the array **a**.

The third line of each test case contains a single integer **q (1 <= q <= 2 * 10^5)** — the number of queries.

The next **q** lines contain two integers each, **l** and **r (1 <= l < r <= n)** — the boundaries of the query.

Output:

For each query, output two integers separated by space: **i** and **j (l <= i, j <= r)**, for which **a_i != a_j**. If such a pair does not exist, output **i = -1** and **j = -1**.

Code:

Start here | Jump_Game.cpp | Diff_ones.cpp | Search_2D_Matrix.cpp | Daily_Temp.cpp

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cout << "Enter array size: ";
    cin >> n;

    vector<int> a(n);

    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    vector<int> diff(n, -1);

    for (int i = n - 2; i >= 0; i--) {
        if (a[i] == a[i + 1]) {
            diff[i] = diff[i + 1];
        }
        else {
            diff[i] = i + 1;
        }
    }
```

Start here | Jump_Game.cpp | Diff_ones.cpp | Search_2D_Matrix.cpp | Daily_Temp.cpp

```cpp
        }
    }

    int q;
    cout << "Enter number of queries: ";
    cin >> q;

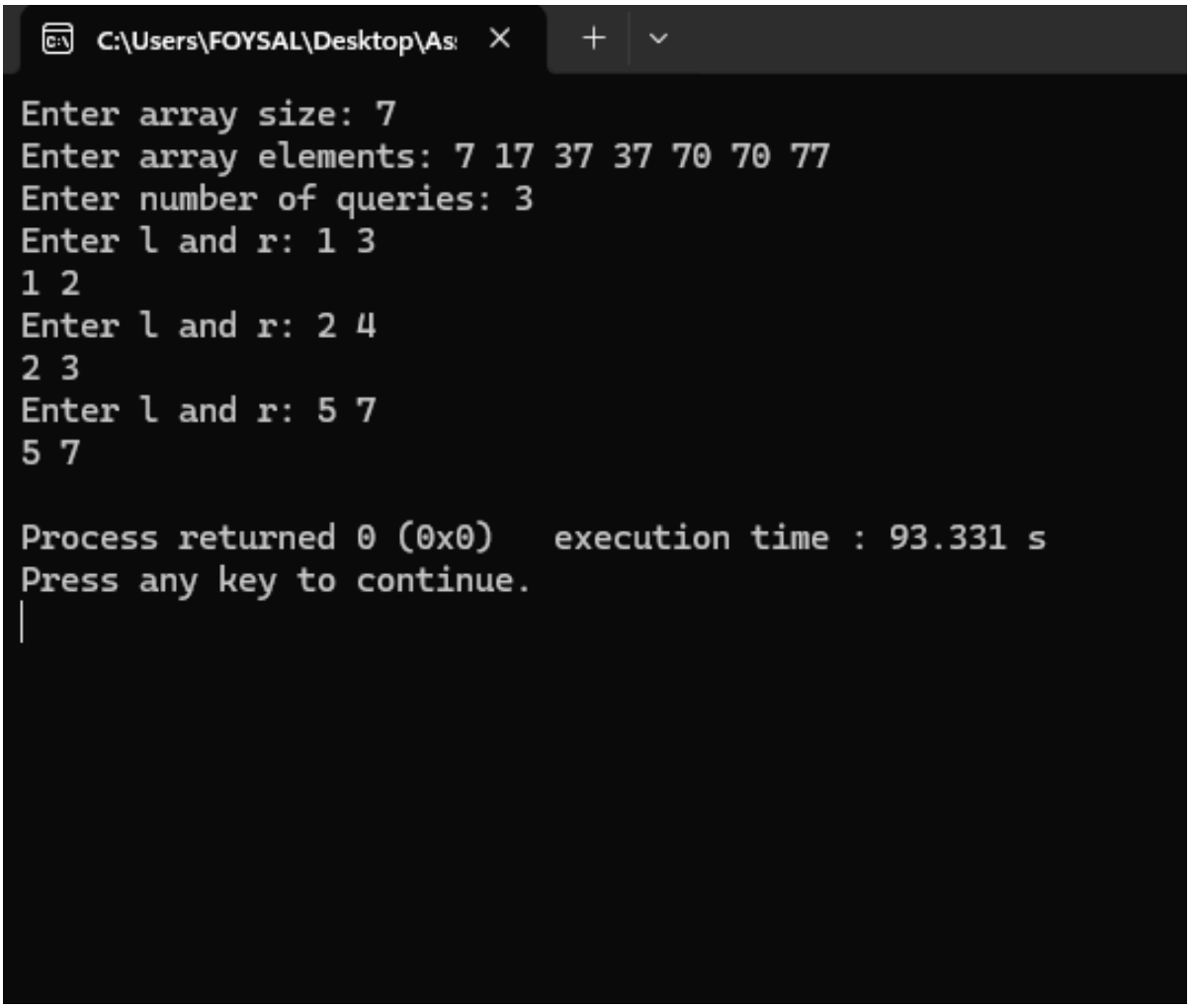
    while (q--) {
        int l, r;
        cout << "Enter l and r: ";
        cin >> l >> r;

        l--;
        r--;

        if (diff[l] != -1 && diff[l] <= r) {
            cout << l + 1 << " " << diff[l] + 1 << endl;
        }
        else {
            cout << "-1 -1" << endl;
        }
    }

    return 0;
}
```

Output:


C:\Users\FOYSAL\Desktop\As
Enter array size: 7
Enter array elements: 7 17 37 37 70 70 77
Enter number of queries: 3
Enter l and r: 1 3
1 2
Enter l and r: 2 4
2 3
Enter l and r: 5 7
5 7
Process returned 0 (0x0)   execution time : 93.331 s
Press any key to continue.

Question 3: Search a 2D Matrix II (LeetCode 240).

Write an efficient algorithm that searches for a value **target** in an **m x n** integer matrix **matrix**. This matrix has the following properties:

- Integers in each row are sorted in ascending from left to right.
- Integers in each column are sorted in ascending from top to bottom.

Example 1:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 5

Output: true

Example 2:

1	4	7	11	15
2	5	8	12	19
3	6	9	16	22
10	13	14	17	24
18	21	23	26	30

Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],[18,21,23,26,30]], target = 20

Output: false

Constraints:

- **m == matrix.length**
- **n == matrix[i].length**
- **1 <= m, n <= 300**
- **-10^9 <= matrix[i][j] <= 10^9**
- All the integers in each row are **sorted** in ascending order.
- All the integers in each column are **sorted** in ascending order.
- **-10^9 <= target <= 10^9**

Code:

Start here | Jump_Game.cpp | Diff_ones.cpp | Search_2D_Matrix.cpp | Daily_Temp.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int m, n;

    cout << "Enter rows and columns: ";
    cin >> m >> n;

    int matrix[100][100];

    cout << "Enter matrix elements:" << endl;
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            cin >> matrix[i][j];
        }
    }

    int target;
    cout << "Enter target: ";
    cin >> target;

    int row = 0;
    int col = n - 1;
    bool found = false;

    while (row < m && col >= 0) {
```

Start here | Jump_Game.cpp | Diff_ones.cpp | Search_2D_Matrix.cpp | Daily_Temp.cpp

```cpp
    while (row < m && col >= 0) {

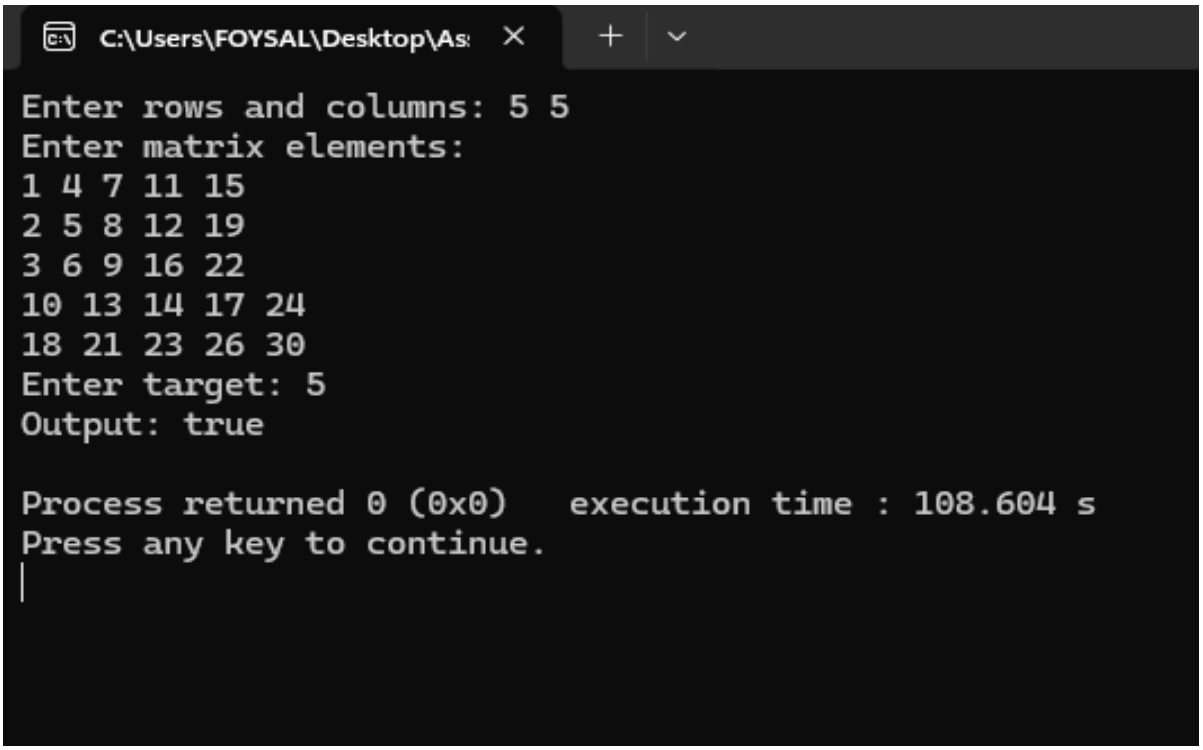
        int current = matrix[row][col];

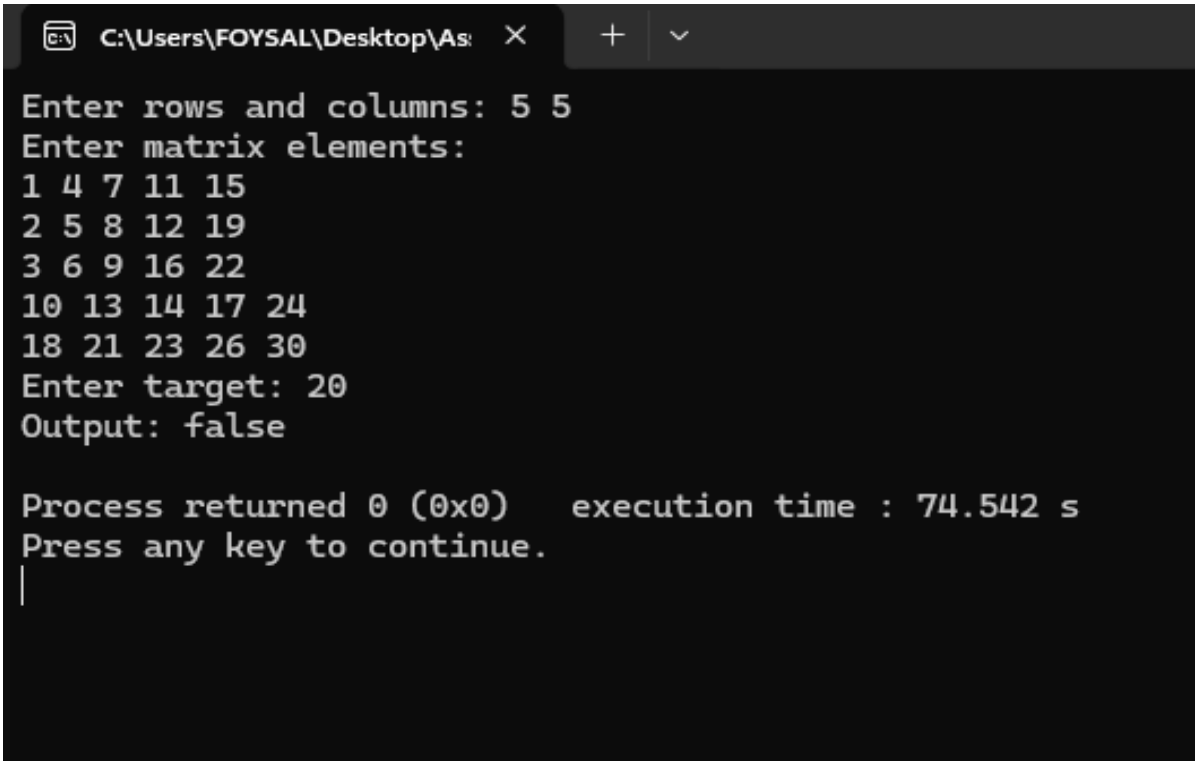
        if (current == target) {
            found = true;
            break;
        }
        else if (current > target) {
            col--;
        }
        else {
            row++;
        }
    }

    if (found) {
        cout << "Output: true" << endl;
    }
    else {
        cout << "Output: false" << endl;
    }

    return 0;
}
```

Output:


C:\Users\FOYSAL\Desktop\As
Enter rows and columns: 5 5
Enter matrix elements:
1 4 7 11 15
2 5 8 12 19
3 6 9 16 22
10 13 14 17 24
18 21 23 26 30
Enter target: 5
Output: true
Process returned 0 (0x0) execution time : 108.604 s
Press any key to continue.


C:\Users\FOYSAL\Desktop\As
Enter rows and columns: 5 5
Enter matrix elements:
1 4 7 11 15
2 5 8 12 19
3 6 9 16 22
10 13 14 17 24
18 21 23 26 30
Enter target: 20
Output: false
Process returned 0 (0x0) execution time : 74.542 s
Press any key to continue.

Question 4: Daily Temperatures (LeetCode 739).

Given an array of integers **temperatures** represents the daily temperatures, return an array **answer** such that **answer[i]** is the number of days you have to wait after the **i^th** day to get a warmer temperature. If there is no future day for which this is possible, keep **answer[i] == 0** instead.

Example 1:

★ **Input:** temperatures = [73,74,75,71,69,72,76,73]

★ **Output:** [1,1,4,2,1,1,0,0]

Example 2:

★ **Input:** temperatures = [30,40,50,60]

★ **Output:** [1,1,1,0]

Example 3:

★ **Input:** temperatures = [30,60,90]

★ **Output:** [1,1,0]

Constraints:

➔ 1 <= temperatures.length <= 10^5

➔ 30 <= temperatures[i] <= 100

Code:

Start here | Jump_Game.cpp | Diff_ones.cpp | Search_2D_Matrix.cpp | Daily_Temp.cpp

```cpp
#include <iostream>
#include <vector>
#include <stack>
using namespace std;

int main() {
    int n;

    cout << "Enter number of days: ";
    cin >> n;

    vector<int> temperatures(n);

    cout << "Enter temperatures: ";
    for (int i = 0; i < n; i++) {
        cin >> temperatures[i];
    }

    vector<int> answer(n, 0);
    stack<int> st;

    for (int i = 0; i < n; i++) {

        while (!st.empty() && temperatures[i] > temperatures[st.top()]) {

            int prevIndex = st.top();
            st.pop();
```

Start here | Jump_Game.cpp | Diff_ones.cpp | Search_2D_Matrix.cpp | Daily_Temp.cpp

```cpp
    vector<int> answer(n, 0);
    stack<int> st;

    for (int i = 0; i < n; i++) {

        while (!st.empty() && temperatures[i] > temperatures[st.top()]) {

            int prevIndex = st.top();
            st.pop();

            answer[prevIndex] = i - prevIndex;
        }

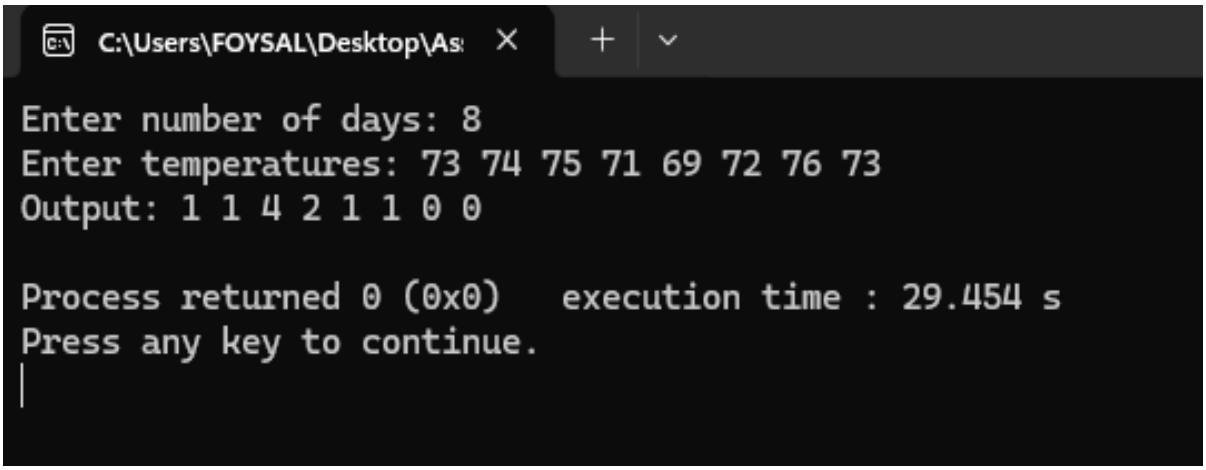
        st.push(i);
    }

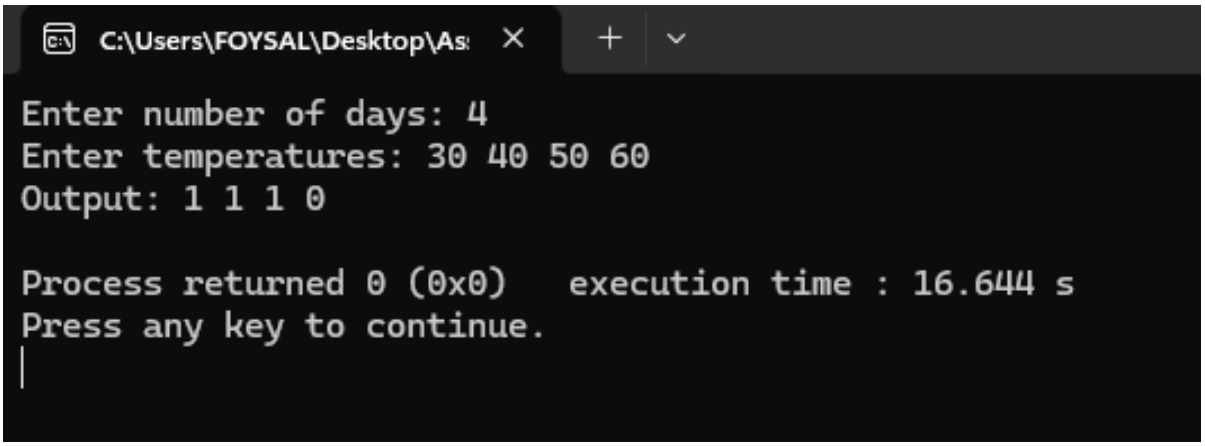
    cout << "Output: ";
    for (int i = 0; i < n; i++) {
        cout << answer[i] << " ";
    }

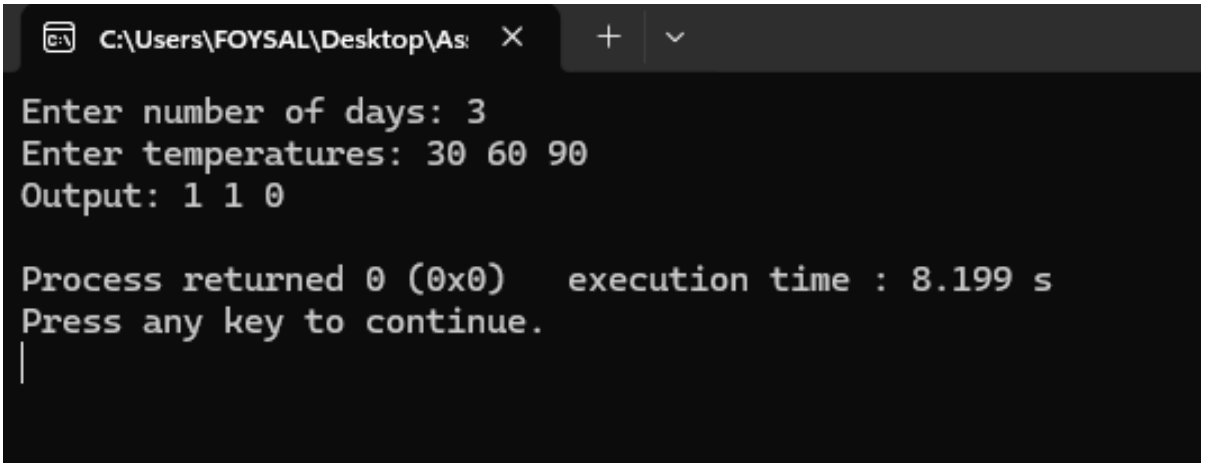
    cout << endl;

    return 0;
}
```

Output:


C:\Users\FOYSAL\Desktop\As
Enter number of days: 8
Enter temperatures: 73 74 75 71 69 72 76 73
Output: 1 1 4 2 1 1 0 0
Process returned 0 (0x0) execution time : 29.454 s
Press any key to continue.


C:\Users\FOYSAL\Desktop\As
Enter number of days: 4
Enter temperatures: 30 40 50 60
Output: 1 1 1 0
Process returned 0 (0x0) execution time : 16.644 s
Press any key to continue.


C:\Users\FOYSAL\Desktop\As
Enter number of days: 3
Enter temperatures: 30 60 90
Output: 1 1 0
Process returned 0 (0x0) execution time : 8.199 s
Press any key to continue.

University of Information Technology & Sciences (UITS)

Department of Computer Science & Engineering

BSc. in CSE — 2nd Year, 1st Semester | Autumn 2026

CSE0613212: Data Structures and Algorithms I Lab

Justification File

Assignment Title: Implementation of Selected Problems Jump Game, Find the Different Ones, Search a 2D Matrix II, and Daily Temperatures Using C++.

1. Skill Mapping:

No.	COs	POs	CP/WP	CA/EA	KP/WK
CO1	Demonstrate knowledge of fundamental data structures and algorithms.	1	-	-	2, 3, 4
CO2	Implement fundamental data structures and algorithms using programming tools to solve specified computational problems.	5	-	-	6
CO3	Analyze problem requirements and justify the selection of appropriate data structures and algorithms.	2	-	-	3, 4
CO4	Design and develop algorithmic solutions for real-world problems using appropriate fundamental data structures and algorithms.	3	1, 2, 3	-	5

2. Justification of Program Outcome (PO) through Course Outcome (CO):

CO	PO	KP	Justification
CO1	PO1 (Engineering Knowledge)	K2, K3, K4	Applies the fundamentals of arrays, 2D arrays (matrices), vectors and stacks, and their operations, to the four problems. The time complexity of the solutions is O(n), O(n + q), O(m + n) and O(n) (Questions 1 to 4).
CO2	PO5 (Modern Tool Usage)	K6	Implements all four solutions in C++ using a programming tool (IDE and compiler) and tests them with the given examples. The code and output screenshots are given for every question (Code and Output of Questions 1 to 4).

CO	PO	KP	Justification
CO3	PO2 (Problem Analysis)	K3, K4	Analyzes the requirements and constraints of each problem and justifies the data structure used: greedy farthest-reach on an array (Q1), a precomputed vector of next different index (Q2), staircase search on the sorted matrix (Q3) and a stack for the next warmer day (Q4).
CO4	PO3 (Design and Development of Solutions)	K5	Designs complete algorithmic solutions for the four real-world style problems, with input, processing and output, and verifies each one on the given examples (Questions 1 to 4).

3. Justification of Complex Engineering Problem (P) and Knowledge Profile (K):

Attributes	Justification
P1: Depth of Knowledge Required	Needs in-depth knowledge of arrays, matrices, vectors and stacks (K3, K4) and the design of efficient algorithms (K5) instead of simple brute-force solutions. (Questions 1 to 4)
P2: Range of Conflicting Requirements	Efficiency versus simplicity, and time versus memory. An extra vector (Q2) and a stack (Q4) are used to save time, and the large input limits make brute force impractical. (Questions 2 and 4)
P3: Depth of Analysis Required	No single obvious solution. Brute force, greedy, precomputation, corner search and stack-based approaches are analyzed to choose an efficient one. (Questions 1 to 4)
P4: Familiarity of Issues	Reaching the last index, answering range queries and finding the next warmer day are not routine array exercises and need new ways of thinking. (Questions 1, 2 and 4)
P5: Extent of Applicable Codes	Follows standard C++ practice (STL vector and stack) and standard Big O notation for efficiency analysis. (Code of Questions 2 and 4)
P6: Extent of Stakeholder Involvement and Conflicting Requirements	The problems are real-world style tasks, such as weather prediction and query answering, where users need correct and fast results. (Questions 2 and 4)

Attributes	Justification
P7: Interdependence	Input reading, preprocessing, main logic and output of each program depend on each other and must work together correctly. (Questions 1 to 4)

4. Justification of Knowledge Profile (K):

Knowledge Profile	Justification
K1 – Theory-based natural sciences	Basic principles of computer memory, where array elements are stored in contiguous locations and accessed by index. (Questions 1 and 3)
K2 – Conceptually-based mathematics and formal aspects of computer science	Big O notation and logical reasoning for correctness, such as the farthest reachable index and the sorted order of rows and columns. (Questions 1 and 3)
K3 – Theory-based engineering fundamentals	Theory of linear data structures: arrays, 2D arrays, vectors and stacks (LIFO) with their operations. (Questions 1 to 4)
K4 – Engineering specialist knowledge	Algorithm techniques: greedy approach, preprocessing for range queries, staircase search and the stack-based next greater element method. (Questions 1 to 4)
K5 – Engineering design	Design of each solution from problem to code: input, algorithm, output and testing with examples. (Questions 1 to 4)
K6 – Engineering practice (technology)	C++ language, STL (vector, stack) and an IDE with compiler used to write, run and test the programs. (Code and Output sections)
K7 – Comprehension of engineering in society	Efficient algorithms save time and computing resources, which benefits the users of real software systems. (Questions 2 and 4)